

Self-Designed Evaluators and Warm Memory for Long-Horizon Agents

Saeid Asgari, Emre Kiciman, Leonardo de Oliveira Nunes, Ranveer Chandra

saeidasgari@microsoft.com

Abstract

A tool-using language-model agent deployed over a long horizon of tasks receives no reward: it cannot tell whether it succeeded, so it cannot safely retry, and it cannot label the experience it would need to improve. We present SELFSUITE, in which the agent’s own base model, given only the world’s public materials and no ground truth, designs a small evaluation suite of weighted judges and a grounded per-task brief, freezes it, and deploys it as the controller of a keep-best retry loop and a typed, outcome-tracked memory. On matched five-repeat benchmarks over tau2-bench and AppWorld with GPT-5.5, SELFSUITE scores above the plain agent without using a single label, matches methods given ten expert labels on tau2-bench, and trails Agentic Context Engineering (ACE) on AppWorld, where code execution gives every learner a direct success signal. In an ablation campaign in which label-free baselines ran on the same tasks, SELFSUITE is above label-free ACE in every repeat, and component ablations identify the second attempt gated by the frozen suite as the only component whose removal hurt in every repeat; memory polarity, the task brief and judge decomposition are within noise. Attempt-level analysis explains why: the self-designed suites are conservative, the keep-best rule turns that conservatism into a low-risk retry gain, and a best-attempt oracle gap remains as headroom. Beyond the label-free setting, we ask how much the one human signal a real deployment has would add: the subject-matter expert (SME) or forward-deployed engineer (FDE) who onboards the agent and grades a handful of its early tasks. Simulating that expert with ten labelled onboarding tasks per world, we spend the labels in the two ways an engineer could. Calibrating SELFSUITE’s evaluator through a deterministic, evaluator-only procedure with a strict acceptance gate yields a small but consistent gain; seeding the memory of a strong context-engineering baseline (Agentic Context Engineering, ACE) with the same expert-graded experience lifts that baseline to tie calibrated SELFSUITE. A single-run study on a second model family reproduces the ordering.

1 Introduction

An agent that serves a long stream of tasks in one world could, in principle, get better at that world as it goes: retry when it has failed, and remember what worked. The bottleneck for such self-improvement is verification rather than action selection. At deployment there is no reward function, no hidden test and no human to say whether a trajectory was correct. Without such a signal the agent cannot decide whether to retry, and it cannot tell a useful lesson from a misleading one, so it cannot improve over a long stream of tasks. In practice the signal is obtained by an expert hand-authoring checks (success rubrics, policy assertions, world-state read-backs, unit tests), which is laborious, world-specific and brittle, and is the reason most “self-improving agent” results assume a benchmark that ships ground truth (GT) [57, 84, 88].

We find that the model can author its own verifier, and that in our benchmarks this is enough for the agent to improve on its plain baseline. Given the same public materials a human would read,

the base model designs a small, decomposed evaluation suite, freezes it, and uses it, unchanged and without any ground truth, to gate a retry loop and to label experience for memory. We call the method SELFSUITE (§4); it is the object of study in every experiment that follows, and we evaluate it under a protocol designed to survive run-to-run noise.

A real deployment does, however, have one human signal: the forward-deployed engineer (FDE) or subject-matter expert (SME) who onboards the agent and can realistically grade about ten tasks per world, and nothing afterwards. We therefore also ask how much that expert adds. We simulate the expert with ten labelled onboarding tasks per world and spend them in the two ways an engineer could (§5): as *evaluator calibration*, an evaluator-only procedure that uses six labels to propose a revised folding, aggregation and threshold for SELFSUITE’s suite and the other four as a strict acceptance gate, or as *warm memory*, expert-graded experience that seeds a playbook learner before deployment, which also gives SELFSUITE its strongest, budget-matched baseline. The answer is that the expert’s labels add a little more, in either way.

Contributions. (1) SELFSUITE: a self-designed, frozen, GT-free control suite with a grounded per-task brief, a world-state read-back that judges consume as evidence, a keep-best retry rule, and a typed profit-and-loss (PnL) memory. (2) A matched, five-repeat evaluation of SELFSUITE across two benchmarks and five worlds with a reported noise floor, paired confidence intervals, component ablations, attempt-level diagnostics, and a second-model replication. (3) A simulated-SME study: ten labelled onboarding tasks per world, excluded from every scored set, spent either as a deterministic evaluator-only calibration of SELFSUITE with a strict fit/gate split or as warm memory for the strongest playbook baseline. All learning is GT-free at deployment; benchmark ground truth is used only for offline scoring.

2 Related work

Self-improving agents and experiential memory. A large body of work lets a large language model (LLM) agent improve without weight updates by writing experience into an external memory. Reflexion [57] stores verbal self-reflections between trials; ExpeL [88] extracts insights from successful and failed trajectories; Voyager [67] and Agent Workflow Memory [71] accumulate reusable skills and workflows; generative agents [53], MemGPT [50], MemoryBank [91], A-MEM [74] and Mem0 [13] study how memory is organised, consolidated and retrieved; Buffer of Thoughts [77], Dynamic Cheatsheet [60] and Learn-by-Interact [59] distil reusable problem-solving templates or synthesised experience; Memento [93] and AdaMEM [86] cast memory use as case-based or test-time adaptation. Agentic Context Engineering (ACE) [84] evolves a structured playbook with a generator, reflector and curator, and identifies “context collapse” under monolithic rewriting and “brevity bias” as failure modes; we use its released memory machinery as our strongest baseline. Surveys cover memory mechanisms [18, 87] and self-evolving agents more broadly [2, 19], and Evo-Memory [72] benchmarks test-time learning over task streams. The learning signal is the common weak point. Most of these methods read environment success, unit tests or benchmark labels; ReasoningBank [49] replaces them with an LLM judge’s self-assessment of each trajectory, and ACE can run without labels by relying on execution feedback and its reflector’s own judgement; and Asadolahi et al. [4] show that such self-assessed scores inflate on incorrect episodes and that the inflation compounds through retrieval. SELFSUITE targets exactly this gap: the signal that labels memory and gates retries is a designed, frozen, state-grounded suite, and every lesson carries a ledger of the outcomes that followed its use.

Prompt and context optimisation. Automatic prompt engineering [95], optimisation by prompting (OPRO) [76], Promptbreeder [20], DSPy [32] and its instruction optimiser [48], TextGrad [82] and GEPA, a genetic-Pareto reflective prompt optimiser [1], optimise prompts or programs against a metric computed on labelled examples; see Mei et al. [44] for a survey of context engineering. These methods assume the metric is available during optimisation. At deployment our agent has no metric, so we optimise nothing online: the evaluator is authored once, frozen, and at most recalibrated from ten expert labels before deployment.

LLM-as-a-judge and self-evaluation. LLM judges are now standard for scoring open-ended outputs [22, 33, 39, 90], with panels [63] and debate [8] used to reduce individual judge bias. Self-feedback loops such as Self-Refine [42], CRITIC [21] and Self-Debug [11], AI feedback [5], self-rewarding models [81], self-taught evaluators [68] and generative verifiers [83] use a model’s own judgements to improve its outputs, building on evidence that models are partly calibrated about

their own correctness [28]. The limits are well documented: without external feedback, intrinsic self-correction often fails to improve reasoning [25, 29], self-refinement amplifies self-bias [73], and LLM evaluators recognise and favour their own generations [52]. Our judges are the agent’s own model, so these concerns apply directly. SELFSUITE mitigates rather than removes them in three ways: judges are decomposed into narrow criteria with a veto rule, they read the resulting world state rather than only the transcript, and they only gate a retry, which keep-best selection adopts only when the suite prefers it by a margin.

Rubrics, checklists and co-evolving evaluators. Decomposing a judgement into explicit criteria improves evaluation, from atomic fact checking [46] to generated checklists [15] and calibrated multi-dimensional rubrics [24]. Rubrics and checklists are increasingly used as rewards for training beyond verifiable domains [23, 26, 37, 64], with expert-written rubrics as evaluation targets in domains such as health [3]; see [10] for a survey. Recent work generates instance-specific rubrics without human annotation [12, 70], adapts rubrics to agent tasks [17], and co-evolves the evaluator with the policy during training [34, 61, 66] or evolves evaluation metrics alongside agent skills under a small reference benchmark [85]. These methods produce training rewards for a policy that is being optimised. SELFSUITE instead has the deployed agent’s base model author one world-level suite from public materials, including the per-task brief policy, aggregation rule and retry policy, and freeze it for deployment; the model’s weights never change. In our study, an online variant that kept revising the rubric from disagreement mining did not improve on the frozen suite (§8).

Evaluating agent trajectories. Automatic evaluators for agents range from vision-language evaluators that also drive refinement [51] and agentic judges that inspect intermediate work [97] to judges that actively query the environment for evidence [56]. Studies of these evaluators find that LLM judges often accept failed web trajectories [41], that reported web-agent progress is overestimated [75], that judges miss silent process faults that an outcome-only view cannot see [47], and struggle to localise errors in long traces [16]. Closest to our setting, RubricForge [54] evolves a frozen judging rubric against trajectories labelled with environment reward on τ -bench and roughly halves false passes, and DeepVerifier [65] uses rubric-guided verification to drive test-time self-refinement of research agents. Both rely on labelled trajectories or a failure taxonomy built offline; SELFSUITE authors its suite without labels, and our ten-label calibration only re-folds and re-thresholds judges that the model already wrote. The same asymmetry, that a false pass ships a wrong answer while a false fail only costs a retry, motivates our conservative veto design.

Verification and test-time selection. Verifiers that rank sampled solutions [14, 36], voting [69], generated tests [9] and search over reasoning or action trees [78, 92] convert extra samples into accuracy when a good selector exists. Repeated sampling raises coverage far faster than selection can exploit it [7], compute-optimal test-time scaling depends on verifier quality [58], multiple aspect verifiers can replace a trained reward model [35], and for agents, knowing when to reflect and how to verify and merge rollouts matters [96]. Reinforcement learning from majority votes [98] or from the model’s own confidence [89] removes external rewards from training altogether. Our second attempt is the smallest instance of test-time scaling, and our best-attempt oracle gap is the agent-level analogue of the coverage-versus-selection gap: the remaining headroom is in the selector, not in generation.

Few-label evaluator alignment. EvalGen [55] aligns LLM-generated evaluation assertions with a user’s grades on a handful of outputs, and AutoCalibrate [40] drafts and refines a judge’s scoring criteria from human labels. Our expert calibration follows the same idea under a stricter protocol for deployment: labels are split into fit and gate sets, only the folding, aggregation and threshold of already-authored judges may change, and a proposal is adopted only if it improves agreement on held-out gate labels without increasing false passes or false fails.

Benchmarks and evaluation practice. We build on ReAct [79] agents and evaluate on τ -bench [80] and its dual-control successor τ^2 -bench [6] and on AppWorld [62]; related agent benchmarks include AgentBench [38], WebArena [94] and SWE-bench [27]. Both of our benchmarks ship ground truth, which we use only for offline scoring. Following calls for error bars and variance reporting in LLM evaluation [43, 45] and for cost-aware, reproducible agent evaluation [30, 31], we report five matched repeats, paired intervals and the run-to-run noise floor.

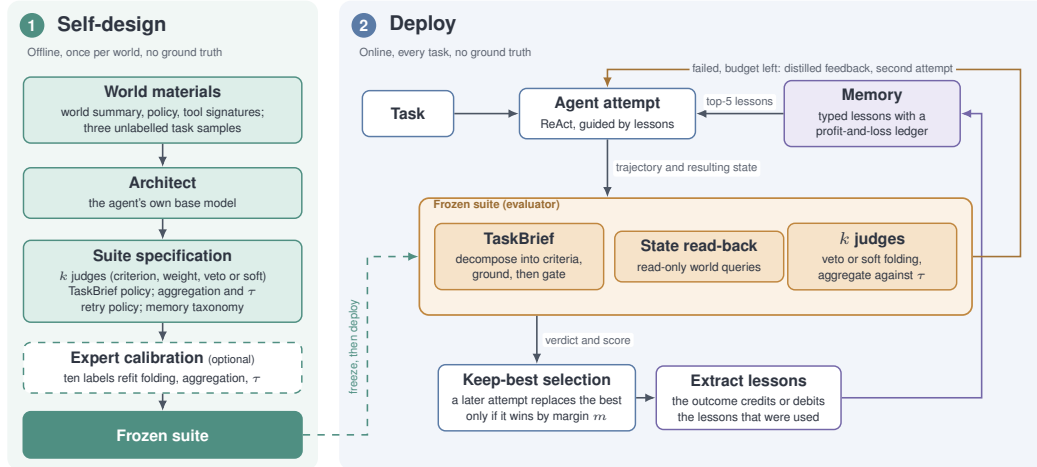


Figure 1: SELF SUITE. Left: offline, once per world and without ground truth, the agent’s base model (the *architect*) authors a suite specification from the world’s public materials; an optional calibration from the expert’s ten onboarding labels may revise its folding, aggregation and threshold; the suite is then frozen. Right: online, for every task, the agent attempts the task guided by retrieved lessons; the frozen suite grounds a per-task brief, reads the resulting world state back, and scores the attempt with k judges; a failed attempt with budget left is retried once from distilled feedback; keep-best selection chooses the attempt to submit; lessons are extracted and the outcome updates the memory ledger.

3 Problem setting and the ground-truth boundary

An agent solves a stream of tasks $x_1, \dots, x_T$ in a world $\mathcal{W}$ that exposes a public summary, a policy document, tool signatures, and a set of read-only queries over its state. Each attempt produces a trajectory τ and a resulting world state. Benchmark ground truth $y_t \in \{0, 1\}$ exists but is available only to the offline scorer.

We distinguish three regimes, and every reported row is labelled with one. *Deployable, GT-free*: the method never reads y_t for any learning or control-flow decision. *Deployable, expert-onboarded*: the method may read y_t for exactly ten reserved onboarding tasks per world, once, before deployment, standing in for the labels an FDE or SME would provide; those tasks are excluded from every scored set. *Oracle ceiling*: the method reads y_t on every task; it is reported only to bound what any signal could buy and is not deployable. Memory is reset at the start of every repeat and is cumulative within a repeat.

4 SELF SUITE: a self-designed, frozen, ground-truth-free control suite

SELF SUITE has two phases (Figure 1): a one-time *self-design* of the evaluator per world, and an online *deploy* loop in which the frozen evaluator controls retries and labels experience. Both phases are ground-truth-free.

Design goals. The design answers five needs of a real deployment. (i) *No hand-authored checks*: the evaluator is authored from the same public materials an engineer would read, so a new world costs one design call rather than a rubric-writing effort. (ii) *Grounding in the world, not the transcript*: judges read the resulting state back, so “the transcript looks right” is not enough to pass. (iii) *Conservative by construction*: veto folding and grounded briefs make false passes rare, which lowers the risk of an automatic retry. (iv) *Outcome-tracked memory*: every lesson carries a ledger, so lessons that precede failures are evicted instead of accumulating. (v) *Frozen and auditable*: the suite does not drift during deployment, its judges can be inspected, and it can be calibrated from a handful of expert labels (§5).

4.1 Self-design: the architect authors the suite

The architect is the agent’s own base model. It receives the world summary, the public policy, the tool signatures and, in the *fresh-suite* treatment, three randomly sampled task descriptions without

labels, and it emits one JSON suite specification with five parts.

Judges. Each judge is one measurement criterion: a system prompt, a user template over the slots `{task, trajectory, policy, tools, lessons, brief, state}`, an output schema that yields a boolean `pass`, a score in $[0, 1]$ and a `reason`, a weight, and a *folding* mode. A VETO judge is a hard gate: if it fails, the attempt fails. A SOFT judge contributes its score to a weighted aggregate. Authored suites are small (one to three judges in our campaigns); typical judges are a final-state outcome verifier, a policy or precondition guard, and a side-effect or scope guard (Appendix B).

TaskBrief policy. Judges need to know what “done” means for the task at hand. The brief policy decomposes each task instruction into success criteria, required evidence, ordering constraints and constraints, each with a supporting quote from the instruction. Two checks keep the brief honest. *Grounding* asks the model whether each item is supported by the public materials and drops any that is not, so judges cannot invent or relax requirements. *Gating* is a narrow precondition check (did each required action occur at all?) that is conservative by construction: it never fabricates a false negative. The grounded brief conditions every judge through the `{brief}` slot.

Aggregation and threshold. The suite names an aggregation rule (`min`, `mean` or `weighted`) and a pass threshold τ . An attempt passes if no veto judge fails and the aggregate of the soft scores is at least τ .

Memory taxonomy and retry policy. The architect names a small set of task families used to route lessons, and a retry policy (number of attempts and how feedback is formed).

The engine supports an optional self-critique pass over the specification; every configuration in this paper freezes the initial authored suite directly. Judges are independent temperature-0 calls to the agent’s own base model; there is no separate, stronger grader.

4.2 State read-back: evidence, not verdict

Transcripts are easy to grade leniently; world state is not. After every attempt, the world adapter reconstructs the resulting environment (`tau2` replays the final state of the simulation; AppWorld reopens the saved task databases) and runs bounded read-only queries over the records the task concerns, serialising the observations into the `{state}` slot. The read-back never touches benchmark reward, gold actions or hidden tests, and it cannot pass, fail or veto anything by itself: state-conditioned judges interpret it relative to the task and the brief. If the read-back fails, state-conditioned judges abstain neutrally (score 0.5, flagged `no_state`) so a missing observation never creates a false veto. All reported suites condition every judge on the read-back; runtime abstentions were 0% on AppWorld normal and below 1% on AppWorld challenge.

4.3 Deploy: retrieve, attempt, judge, retry with keep-best, learn

For each task the solver runs the following steps.

1. **Retrieve.** The task is routed to a family and the top-5 relevant lessons are retrieved: family-first gather, presort by anchor status and recency, then an LLM relevance rank over a window of about twenty candidates.
2. **Attempt.** The ReAct agent runs with the retrieved lessons in its prompt.
3. **Judge.** The frozen suite scores the attempt: the brief is built and grounded, the state is read back, each judge emits `pass` and `score`, and the verdicts are folded (vetoes first, then the aggregate against τ).
4. **Retry.** If the attempt failed and budget remains (two attempts in total), the failing judges’ reasons are distilled into one feedback instruction and the agent attempts the task again with that instruction. The feedback describes what the world evidence showed; it never mentions judges or criteria.
5. **Keep-best.** The attempt to submit is chosen by a do-no-harm rule: a suite-passing attempt wins; ties go to the earliest attempt; a later attempt replaces the current best only if its aggregate score exceeds the best by a margin m ($m = 0.30$ in the repeated campaigns, 0.15 in the single-run study). A noisy second attempt therefore cannot displace a good first one.
6. **Learn.** Lessons are extracted from the completed task and written to memory, and the outcome updates the ledger of every lesson that was retrieved for the task.

Algorithm 1 Warm-up-v2 evaluator calibration (six fit tasks, four gate tasks)

Require: authored suite S_0 ; ten labelled warm-up tasks with recorded per-judge verdicts on fixed attempts

- 1: Split the ten task identifiers by a seeded shuffle into F (six, fit) and G (four, gate)
- 2: On F , compute each judge’s fire rate given correct and given wrong attempts; a judge is *eligible* if $\Pr[\text{fire} \mid \text{wrong}] - \Pr[\text{fire} \mid \text{correct}] \geq 0$
- 3: Enumerate candidates: eligible subset $\times$ folding profile $\in \{\text{preserve, state-veto, all-soft}\} \times$ aggregation $\in \{\text{min, mean, weighted}\} \times$ threshold grid (for min, only τ_0)
- 4: Keep candidates whose agreement and balanced accuracy on F do not fall below S_0 ; propose the candidate maximising (balanced accuracy, agreement, $-\text{false positives}$, $-\text{false negatives}$, $-\#\text{judges}$, $-|\tau - \tau_0|$)
- 5: **Accept** the proposal only if, on G , agreement strictly improves and neither false positives nor false negatives increase (and the F metrics did not decrease); **otherwise keep** S_0
- 6: Freeze the resulting suite for the entire deployment

4.4 Typed, outcome-tracked memory

Lessons are typed (positive “do” or negative “avoid”), sectioned (strategies, common mistakes, domain knowledge), routed to the architect’s families, and stored in per-bucket-capped lists: at most 30 entries per bucket, LLM distillation of near-duplicates when a bucket reaches 20, and a hard global bound of 200 entries per scope. Every entry carries a PnL ledger with `uses`, `helps` and `hurts` counters: a lesson retrieved before a success is credited and before a failure debited, so persistently harmful lessons are down-weighted and become eligible for eviction. Only five relevance-ranked entries are injected per task, in contrast to a playbook injected as a block. The main treatment uses *balanced* memory (positive and negative lessons); polarity variants are ablated in §7.2.

5 Simulating the forward-deployed engineer: ten labels, two ways to spend them

SELFSUITE needs no labels. A real deployment nevertheless has one human signal, and it is natural to ask what it adds: the agent is onboarded by a forward-deployed engineer or a subject-matter expert who knows the world’s policy, watches the agent on a few tasks, and says which ones were done right. We simulate that expert with ten labelled onboarding tasks per world: the agent attempts them once, the expert’s verdict on each attempt is the benchmark label, and those ten tasks are then excluded from every scored set. Ten is a budget an engineer can actually deliver during onboarding; it is far below what any training procedure needs. The expert’s experience can be spent in two ways.

5.1 Calibrating the self-designed evaluator

The calibration (Algorithm 1) is deterministic and *evaluator-only*: it may change which authored judges are kept, their folding, the aggregation rule and the threshold, and nothing else. The actor, the pre-action briefing setting (off), the brief and its gate, the memory and retry policies, the selection margin and the state signal are held fixed, and the ten warm-up tasks are excluded from every scored set.

The calibration operates on *recorded* verdicts: the authored suite is run once on the ten warm-up tasks, every judge’s verdict is stored, and candidate suites are re-aggregated from those verdicts without re-running the agent or the judges. The eligibility rule encodes the principle that a veto must be discriminative: a judge that fires on correct attempts as often as on wrong ones only adds retries. With four gate tasks the acceptance rule is deliberately conservative, and it has two consequences. First, the judges only gate retries, so a better evaluator cannot move ground-truth accuracy unless the agent produces a better candidate to select. Second, in the main campaign only the retail proposal cleared the gate (gate agreement $66.7\% \rightarrow 83.3\%$, a policy-guard judge dropped); the airline proposal would have reduced gate agreement and was rejected; telecom and both AppWorld suites already agreed with all four gate labels, so their authored suites were kept (Appendix A).

5.2 Seeding memory with expert-graded experience

The other way to spend the expert’s labels is as *warm memory*: the agent’s onboarding attempts, together with the expert’s verdict on each, are reflected into the agent’s playbook before deployment, so the agent starts its label-free stream with lessons that were graded by someone who knew the answer. We implement this with ACE [84]: a warm playbook is fitted with ACE’s Reflector and Curator from one binary outcome label per onboarding task, after which deployment continues label-free with ACE’s own trace self-inference. (Grading ACE’s deployment trajectories with the calibrated suite instead of its own trace self-inference is a further arm, reported in Appendix C.)

6 Experimental setup

Benchmarks and metrics. tau2-bench [6] airline (50 tasks), retail (114) and telecom (114): conversational tool-calling agents with a simulated user, scored by pass@0.5 (best-trial reward ≥ 0.5). AppWorld [62] test-normal (168) and test-challenge (417): code-generating agents over app APIs, scored by Task Goal Completion (all subtasks pass, reward ≥ 0.999). A task whose run directory has no evaluation report counts as a failure.

Models. All repeated campaigns use GPT-5.5 at low reasoning effort. The single-run cross-model study adds Claude Opus-4.8 with thinking off. Models are reached through an internal gateway; the judge model is always the agent’s own model.

Baselines. A plain ReAct [79] **agent**. **ACE**, driven around our rollouts with the authors’ released playbook utilities, the released no-GT Reflector and Curator prompts and delta (accumulating) memory, under a strict no-GT rule: the released code’s `no_ground_truth` only hides the target from the Reflector prompt but still gates control flow on correctness, whereas ours never reads benchmark reward for any learning or control decision. **ACE + warm playbook** (ten labels). In the single-run study the ACE reference is that campaign’s own strict no-GT implementation.

Campaigns. (A) *Matched five-repeat benchmark*: five repeats of 25 tau2 tasks (8 airline; 8 or 9 retail; 9 or 8 telecom) plus 25 AppWorld tasks (12 or 13 from normal and 13 or 12 from challenge, alternating); task identifiers are identical across every method, disjoint across repeats, and disjoint from the ten reserved onboarding tasks per world (airline’s 40 evaluation plus 10 onboarding tasks exhaust its catalog). The AppWorld group therefore mixes the two splits; the campaign records the group totals. (B) *Ablation campaign*: five matched repeats with the same SELFSUITE configuration as (A) (balanced memory, two attempts, margin 0.30) as reference, seven single-component variants, and the plain agent and ACE without labels (the authors’ implementation on AppWorld, our re-implementation on tau2) on the same tasks. Each repeat has 18 or 19 public tau2 tasks (6 or 7 per domain) and 25 AppWorld tasks (12 or 13 normal, 13 or 12 challenge); the campaign also scored a non-public experimental tau2 domain, which we exclude. Per-stratum results separate AppWorld normal from challenge. (C) *Two-model single-run study*: one run per cell on the full suites for GPT-5.5 (five worlds) and Opus-4.8 (four worlds; its AppWorld-challenge run was invalidated by gateway failures). (D) *Suite diagnostics*: attempt-level records from the single GPT-5.5 fresh-suite runs of (C) and from five independent deployments of the calibrated suites on every catalog task of airline, retail, telecom and AppWorld normal (the ten onboarding tasks excluded), used in §7.3 to characterise how the suites behave rather than to rank methods.

Hyperparameters. Fresh suites are designed from three unlabelled sampled task descriptions; two attempts per task; selection margin 0.30; balanced memory; briefing off; revision off; state signal required; top-5 retrieval; judge temperature 0; distinct deterministic seeds per repeat and world.

Statistics. We report mean and sample standard deviation (SD) over repeats, paired per-repeat differences with a 95% *t* confidence interval (CI) and a paired *t*-test in the matched campaign, Welch’s *t*-test for unpaired comparisons, following recommendations to report uncertainty in LLM evaluation [43, 45]. The observed SD of a 50-task cell is 2 to 7 pp and of a 25-task tau2 slice 4 to 13 pp; every comparison below should be read against that floor.

7 Results

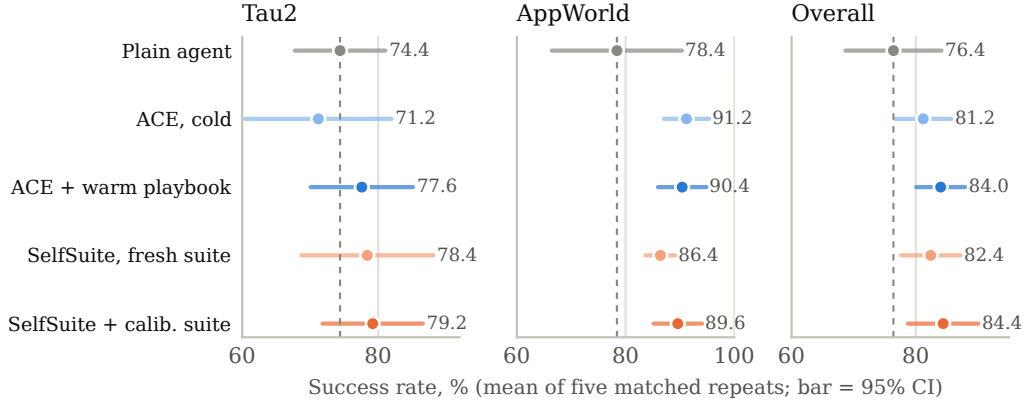


Figure 2: Matched five-repeat benchmark (campaign A), GPT-5.5. Mean success over five matched repeats with 95% CI; the dashed line marks the plain agent. Methods separate on tau2 (policy compliance with no execution signal); on AppWorld the two ACE rows and calibrated SELF SUITE lie within noise of each other, and label-free SELF SUITE is below both ACE rows.

Table 1: Paired per-repeat differences (campaign A). Mean $\pm$ SD of the five per-repeat differences in percentage points, 95% t CI, and paired t -test p -value.

Comparison	Group	Δ (pp)	95% CI, p
SELF SUITE + calibrated — plain agent	overall	$+8.0 \pm 4.7$	$[+2.2, +13.8], 0.019$
	tau2	$+4.8 \pm 4.4$	$[-0.6, +10.2], 0.070$
	AppWorld	$+11.2 \pm 7.7$	$[+1.6, +20.8], 0.031$
SELF SUITE, fresh — plain agent	overall	$+6.0 \pm 5.1$	$[-0.3, +12.3], 0.058$
	ACE + warm playbook — plain agent	$+7.6 \pm 4.3$	$[+2.2, +13.0], 0.017$
ACE, cold — plain agent	overall	$+4.8 \pm 8.6$	$[-5.8, +15.4], 0.278$
SELF SUITE + calibrated — SELF SUITE, fresh	overall	$+2.0 \pm 1.4$	$[+0.2, +3.8], 0.034$
	tau2	$+0.8 \pm 3.4$	$[-3.4, +5.0], 0.621$
	AppWorld	$+3.2 \pm 4.4$	$[-2.2, +8.6], 0.178$
SELF SUITE + calibrated — ACE + warm playbook	overall	$+0.4 \pm 2.6$	$[-2.8, +3.6], 0.749$
	tau2	$+1.6 \pm 5.4$	$[-5.1, +8.3], 0.541$
	AppWorld	-0.8 ± 3.4	$[-5.0, +3.4], 0.621$
ACE + warm playbook — ACE, cold	overall	$+2.8 \pm 5.9$	$[-4.6, +10.2], 0.351$
	tau2	$+6.4 \pm 9.2$	$[-5.0, +17.8], 0.195$

7.1 Matched five-repeat benchmark

Figure 2 gives the headline comparison (the mean and SD of every arm is listed in Table 3, Appendix C); Table 1 gives the paired differences. Four observations. First, SELF SUITE with a fresh suite and no labels is above the plain agent on average ($+6.0$ pp paired, $[-0.3, +12.3]$) and within noise of every learner on tau2, where it has the second-best mean of any method; on AppWorld it is below both ACE rows (paired -4.8 and -4.0 pp, $p = 0.03$ each). Second, calibrated SELF SUITE has the best mean overall and on tau2 (84.4% and 79.2%) and ties ACE seeded with expert-graded warm memory, its strongest budget-matched baseline (84.0%; paired difference $+0.4 \pm 2.6$ pp). Third, the expert’s labels help in both uses: as warm memory they lift ACE from 81.2 to 84.0 ($+2.8 \pm 5.9$ pp overall and $+6.4 \pm 9.2$ on tau2, not significant with five repeats), and as evaluator calibration they are worth a paired $+2.0$ pp to SELF SUITE ($[+0.2, +3.8]$), the smallest and most consistent difference in the campaign (per-repeat differences $+2, +2, +2, 0, +4$; the calibrated suite never loses a repeat). Fourth, the benchmarks behave differently: on tau2, where policy compliance has no execution signal, the two SELF SUITE rows lead (78.4 and 79.2 versus 71.2 to 77.6) and the methods spread over 8 pp; on AppWorld, where code execution hands every learner a free success signal, both ACE rows and calibrated SELF SUITE lie between 89.6 and 91.2 and within noise of each other, while the fresh suite is the lowest learner (86.4, 5.6 pp below the best, Welch $p = 0.06$).

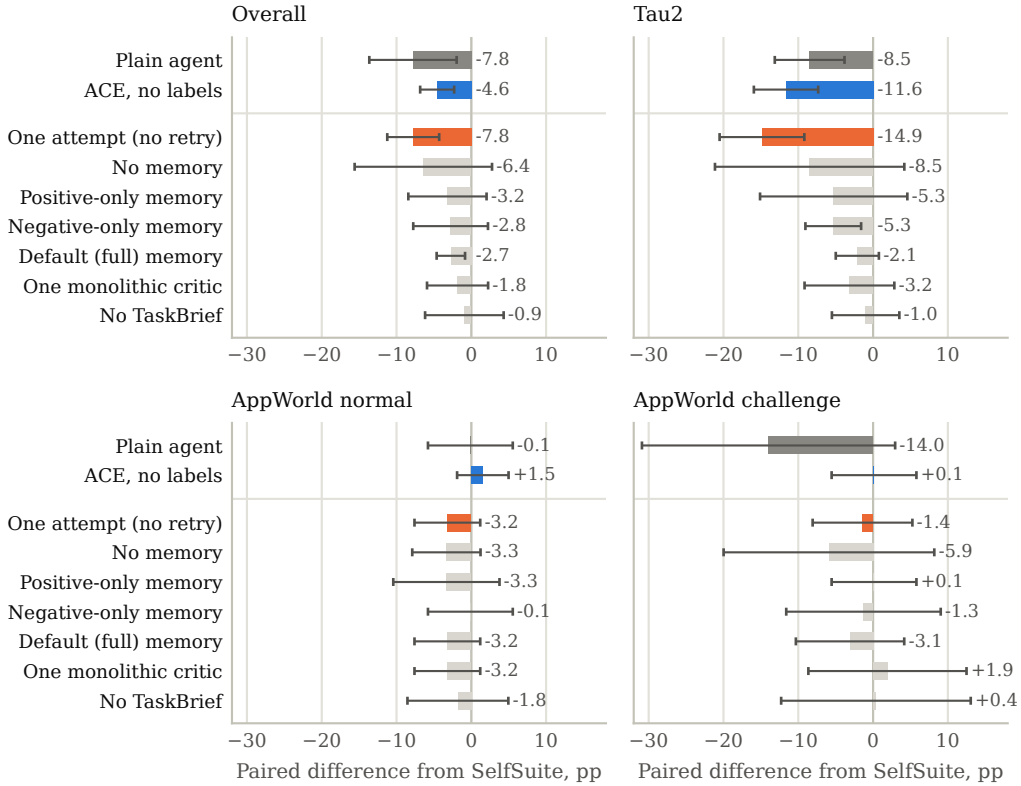


Figure 3: Component ablations and label-free baselines (campaign B, GPT-5.5). Paired per-repeat difference from SELFSUITE (balanced memory, two attempts) over five matched repeats of 18 or 19 tau2 and 25 AppWorld tasks (43 or 44 overall); bars are means, whiskers the SD of the five differences. Top: the plain agent and ACE without labels on the same tasks. Bottom: SELFSUITE with one component removed. SELFSUITE scores: overall 89.5 ± 5.4 , tau2 86.2 ± 7.0 , AppWorld normal 95.4 ± 6.9 , AppWorld challenge 88.6 ± 11.1 .

7.2 Which components matter

Figure 3 removes one component at a time from SELFSUITE and, on the same tasks, runs the two label-free baselines. *Baselines.* SELFSUITE scores $89.5 \pm 5.4\%$ overall against $85.0 \pm 4.1\%$ for ACE without labels and $81.7 \pm 4.6\%$ for the plain agent: paired $+4.6 \pm 2.3$ pp over ACE ($[+1.7, +7.4]$, $p = 0.011$, positive in all five repeats) and $+7.8 \pm 5.8$ pp over the agent ($[+0.6, +15.1]$, $p = 0.040$). The margin comes from tau2 (86.2 versus 74.6 and 77.7); on AppWorld the differences are within noise. *Components.* Removing the second attempt gated by the frozen suite costs 7.8 ± 3.5 pp overall ($[-12.1, -3.5]$, $p = 0.007$) and 14.9 ± 5.7 pp on tau2, and is negative in all five repeats in both; on AppWorld its effect is small in both splits (-3.2 normal, -1.4 challenge), consistent with the execution signal that world already provides. Replacing balanced memory by the default memory setting costs a small but consistent 2.7 ± 1.9 pp ($p = 0.032$, never positive), which supports the balanced design. Removing memory costs 6.4 pp on average but with an SD of 9.2 (it helps in two repeats and hurts badly in three). Memory polarity, the TaskBrief and judge decomposition (a single monolithic critic without a brief) are within one SD of zero overall, although negative-only memory is below the reference on tau2 in four of five repeats (-5.3 ± 3.7 pp, $p = 0.033$).

7.3 What the suites do: strictness, retry gain and headroom

Figure 4 explains the ablation result using the attempt-level records of campaign D (exact values in Table 4, Appendix D). *Fresh* suites are strict: on single GPT-5.5 runs they judged only 6% (airline), 17% (AppWorld normal), 37% (telecom) and 64% (retail) of tasks as passed while ground-truth success was 66 to 90%, with only 0 to 10 false-pass attempts per world against dozens to hundreds of false fails. Such a suite retries most tasks (56 to 100%), and because keep-best never lets a second

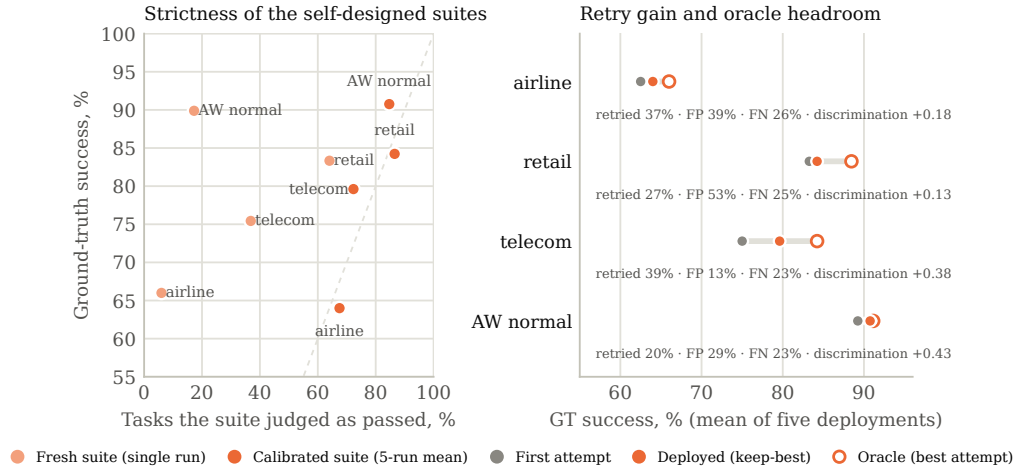


Figure 4: Attempt-level diagnostics. Left: fraction of tasks each suite judged as passed versus ground-truth success. Fresh suites (single runs, GPT-5.5) lie far below the diagonal: they are strict vetoers that rarely false-pass. Calibrated suites (five deployments on the whole task sets, means) sit near the diagonal. Right: for the calibrated suite, first-attempt success, deployed keep-best success and the best-attempt oracle, annotated with the fraction of tasks retried and the per-attempt false-positive (FP) and false-negative (FN) rates and discrimination (mean judge score on GT-pass minus GT-fail attempts) of the suite verdict. Judges, aggregation and thresholds of these suites are listed in Appendix B.

attempt displace a first unless it scores strictly higher by the margin, the retries convert into net gains without net losses: first-attempt success rose from 62.0 to 66.0 (airline), 81.6 to 83.3 (retail) and 70.2 to 76.3 (telecom, full memory), with 3 to 9 tasks gained and 1 to 3 lost per world (AppWorld normal: 90.5 $\rightarrow$ 89.9, one task lost). *Calibrated* suites are more permissive (judged pass rates 67 to 87%, FN rates 23 to 26%, FP rates 13 to 53%) and retry only 20 to 39% of tasks, yet they gain +1.0 to +4.6 pp per world and lose on average less than one task per run. The largest gain, telecom (+4.6 pp), is also the world with the lowest FP rate and the highest discrimination, which is the pattern one expects if the gain is produced by a vetoer that is right about failures.

The remaining gap to the best-attempt oracle (+2.0 airline, +4.3 retail, +4.6 telecom, +0.3 AppWorld normal for the calibrated suite; +2.4 to +12 on the fresh single runs) is selection headroom: the agent already produces a correct attempt that often, and the GT-free suite does not pick it, the agent-level counterpart of the gap between coverage and selection in repeated sampling [7]. Better selection would convert directly into task success, which makes the verifier a natural next target.

7.4 Second model family (single runs)

On a second model family the ordering reproduces (Figure 5; exact values in Table 5, Appendix D): SELFSUITE is best or tied on seven of nine cells, ties retail, and loses airline on both models, with lifts over the agent of +4.9 pp (GPT-5.5) and +8.4 pp (Opus-4.8) against +3.0 and +6.9 for that campaign’s ACE. Because these are single runs they are supporting evidence for the direction of the effects, not for their size. The best-attempt oracle (Figure 5) exceeds the deployed score by 7.4 pp on average and by 18.4 pp on Opus-4.8 telecom, and judge discrimination is positive on eight of nine cells (GPT-5.5: +0.05, +0.09, +0.09, +0.08, +0.29 for AppWorld normal, AppWorld challenge, airline, retail and telecom; Opus-4.8: +0.16, +0.07, +0.25, and -0.01 on telecom, the cell with the largest headroom).

8 Discussion and limitations

What SELFSUITE buys, and how. Without any labels, a frozen self-designed suite with one gated retry puts the agent ahead of the plain agent on average and level with every learner on tau2, the setting with no execution signal, where its mean is above every ACE variant (within noise); on

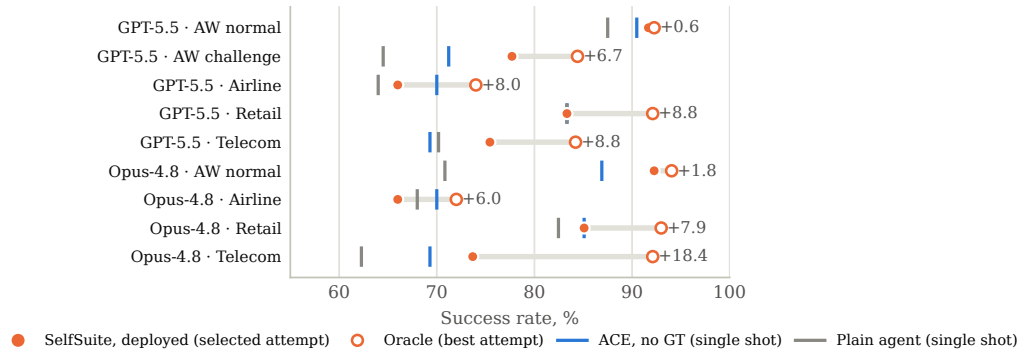


Figure 5: Two-model single-run study and selection headroom (campaign C). One run per cell on the full suites. Deployed SELFSUITE success (single-run configuration: no briefing, no revision, selection margin 0.15), its best-attempt oracle, ACE (that campaign’s strict no-GT implementation) and the plain agent. Opus-4.8 AppWorld challenge is omitted (invalid run). The oracle exceeds ACE in all nine cells and the deployed score by 7.4 pp on average.

AppWorld, where code execution gives ACE a direct signal, it trails ACE. The ablations and the attempt-level analysis agree on the mechanism: the suites are conservative vetoers, keep-best limits the downside of the retry, and the second attempt is the component whose removal hurts in every repeat. In the ablation campaign, where both label-free baselines ran on the same tasks, SELFSUITE is above ACE without labels in all five repeats and above the plain agent on average. The single-run study on two model families gives the same ordering on every world but airline. What remains on the table is selection: the agent produces a correct attempt more often than the suite picks it.

What the expert adds. In the matched campaign only the plain agent is significantly below the best. Calibrated SELFSUITE has the best mean on tau2 and overall, and ACE seeded with expert-graded warm memory, the strongest baseline once it receives the same ten labels, ties it. The two reach the same operating point by different routes: SELFSUITE spends its effort on a second attempt gated by a conservative self-authored judge; the warm-memory learner spends the expert’s labels on a better first attempt.

What the expert’s ten labels buy. Spent as warm memory, the labels buy the larger point gain (+2.8 pp overall and +6.4 pp on tau2 for the playbook learner, not significant with five repeats): experience that was graded by someone who knew the answer is worth more than the same experience self-graded. Spent as evaluator calibration, they buy a small, consistent gain for SELFSUITE (+2.0 pp paired, non-negative in every repeat) and a less variable evaluator; the calibration keeps the authored suite in three of five worlds, so the value of the labels is as much a guarantee as an improvement. The two routes are complementary in principle: nothing prevents seeding SELFSUITE’s memory from the same onboarding attempts, which we leave to future work. Grading ACE’s trajectories with the calibrated suite instead of its own self-inference adds nothing measurable on top of the warm playbook, and on our context-collapsing re-implementation of ACE the evaluator’s effect was to stop drift: a warm playbook that kept adapting label-free lost 3.2 pp on tau2 relative to the same playbook frozen, while adapting under the calibrated evaluator held it (Appendix C).

Why retry, and why it is low-risk. The self-designed suites are conservative vetoers. The keep-best rule limits the downside of retrying: a retry can only replace the first attempt when the suite is more confident in it, so worlds with a discriminative veto (telecom) gain the most and no world loses more than one task per run on average. The same property explains why the judges cannot buy accuracy by themselves: they gate retries; they do not generate candidates.

The signal the world already gives. On AppWorld the ACE rows and calibrated SELFSUITE tie because code execution exposes success to all of them, and label-free SELFSUITE trails ACE; the ablations show the retry effect shrinking to a few points on both AppWorld splits. Claims about GT-free learning are most informative on worlds like tau2, where compliance with a policy leaves no execution trace; that is also where SELFSUITE has its highest means, though within noise of warm ACE.

Airline. SELFSUITE does not beat the plain agent on airline on either model in the single-run study

and trails ACE there, and the calibration could not improve the airline suite on the gate tasks. Airline’s policy is dense enough that a slightly wrong lesson or judge costs more than it saves. We report it as a limit of the approach rather than tune around it.

Limitations. The judge, the brief grounder, the memory ranker and the agent are the same model; decomposition, grounding and state read-back reduce but do not remove self-evaluation circularity [25, 52], and Opus-4.8 telecom shows the residual (negative discrimination with 18 pp of headroom). Fresh suites differ from repeat to repeat, so part of the reported variance is design variance; the calibrated suite removes it within a campaign at the price of freezing whatever the ten warm-up tasks supported. Five repeats bound effects of about ± 4 pp on a 50-task cell; smaller effects (memory polarity, fresh versus calibrated suites on tau2) are not resolved by this study. AppWorld challenge appears in the single-run study and in the ablation campaign but not on its own in the matched campaign, and Opus-4.8 only in the single-run study. The comparison is not cost-normalised: SELFSUITE adds judge, brief, retrieval and retry calls per task, and a per-task token accounting, as advocated for agent evaluation [30, 31], is future work. Finally, an online variant in which the rubric keeps being revised from GT-free disagreement mining did not improve on the frozen calibrated suite in the matched campaign; whether windowed rubric evolution can help with a larger adjudication budget is open.

9 Conclusion

A base model can author its own decomposed, state-grounded evaluation suite, freeze it, and use it with no ground truth to gate a keep-best retry loop and to label memory. Under a matched five-repeat protocol on two benchmarks, that system scores above the plain agent without any labels, level with the labelled methods on tau2 but behind ACE on AppWorld; the retry carries the effect, the retry is low-risk because the suites are conservative, and the remaining headroom is selection. The ten labels a subject-matter expert can provide at onboarding add a little more, in either of two ways: as evaluator calibration they lift SELFSUITE by a small margin that was non-negative in every repeat, and as expert-graded warm memory they lift a strong context-engineering learner to tie it. Replacing hand-authored evaluators with self-designed ones is therefore practical with no labels at all and slightly better with the few an expert actually has; seeding the self-designed system’s memory from the same onboarding experience, and improving its selector, are the next steps.

References

- [1] L. A. Agrawal, S. Tan, D. Soylu, N. Ziems, R. Khare, K. Opsahl-Ong, A. Singhvi, H. Shandilya, M. J. Ryan, M. Jiang, C. Potts, K. Sen, A. G. Dimakis, I. Stoica, D. Klein, M. Zaharia, and O. Khattab. GEPA: Reflective prompt evolution can outperform reinforcement learning. In *International Conference on Learning Representations*, 2026. arXiv:2507.19457.
- [2] H. ang Gao, J. Geng, W. Hua, M. Hu, X. Juan, H. Liu, S. Liu, J. Qiu, X. Qi, Y. Wu, H. Wang, H. Xiao, Y. Zhou, S. Zhang, J. Zhang, J. Xiang, Y. Fang, Q. Zhao, D. Liu, Q. Ren, C. Qian, Z. Wang, M. Hu, H. Wang, Q. Wu, H. Ji, and M. Wang. A survey of self-evolving agents: What, when, how, and where to evolve on the path to artificial super intelligence. *Transactions on Machine Learning Research*, 2026. arXiv:2507.21046.
- [3] R. K. Arora, J. Wei, R. S. Hicks, P. Bowman, J. Quiñonero-Candela, F. Tsimpourlas, M. Sharman, M. Shah, A. Vallone, A. Beutel, J. Heidecke, and K. Singhal. HealthBench: Evaluating large language models towards improved human health. *arXiv preprint arXiv:2505.08775*, 2025.
- [4] M. Asadolahi, A. Amini, S. Talebi, A. Farhadi, and A. Zamanifar. Memory reward inflation in self-improving LLM agents. *arXiv preprint arXiv:2608.00017*, 2026.
- [5] Y. Bai, S. Kadavath, S. Kundu, A. Askell, J. Kernion, A. Jones, A. Chen, A. Goldie, A. Mirhoseini, C. McKinnon, C. Chen, C. Olsson, C. Olah, D. Hernandez, D. Drain, D. Ganguli, D. Li, E. Tran-Johnson, E. Perez, J. Kerr, J. Mueller, J. Ladish, J. Landau, K. Ndousse, K. Lukosuite, L. Lovitt, M. Sellitto, N. Elhage, N. Schiefer, N. Mercado, N. DasSarma, R. Lasenby, R. Larson, S. Ringer, S. Johnston, S. Kravec, S. E. Showk, S. Fort, T. Lanham, T. Telleen-Lawton, T. Conerly, T. Henighan, T. Hume, S. R. Bowman, Z. Hatfield-Dodds, B. Mann, D. Amodei, N. Joseph, S. McCandlish, T. Brown, and J. Kaplan. Constitutional AI: Harmlessness from AI feedback. *arXiv preprint arXiv:2212.08073*, 2022.

- [6] V. Barres, H. Dong, S. Ray, X. Si, and K. Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.
- [7] B. Brown, J. Juravsky, R. Ehrlich, R. Clark, Q. V. Le, C. Ré, and A. Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling. *arXiv preprint arXiv:2407.21787*, 2024.
- [8] C.-M. Chan, W. Chen, Y. Su, J. Yu, W. Xue, S. Zhang, J. Fu, and Z. Liu. ChatEval: Towards better LLM-based evaluators through multi-agent debate. In *International Conference on Learning Representations*, 2024. arXiv:2308.07201.
- [9] B. Chen, F. Zhang, A. Nguyen, D. Zan, Z. Lin, J.-G. Lou, and W. Chen. CodeT: Code generation with generated tests. In *International Conference on Learning Representations*, 2023. arXiv:2207.10397.
- [10] H. Chen, Z. Han, Y. Yan, Q. Zhu, M. Sun, and W. Che. From holistic evaluation to structured criteria: Rubrics across the evolving LLM landscape. *arXiv preprint arXiv:2606.08625*, 2026.
- [11] X. Chen, M. Lin, N. Schärli, and D. Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations*, 2024. arXiv:2304.05128.
- [12] Y. Chen, H. Li, Q. Ai, F. Zhu, T.-S. Chua, M. Zhang, and Y. Liu. GenRubric: Self-evolving rubric generation for scalable LLM evaluation. *arXiv preprint arXiv:2608.29856*, 2026.
- [13] P. Chhikara, D. Khant, S. Aryan, T. Singh, and D. Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [14] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, C. Hesse, and J. Schulman. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- [15] J. Cook, T. Rocktäschel, J. Foerster, D. Aumiller, and A. Wang. TICKing all the boxes: Generated checklists improve LLM evaluation and generation. *arXiv preprint arXiv:2410.03608*, 2024.
- [16] D. Deshpande, V. Gangal, H. Mehta, J. Krishnan, A. Kannappan, and R. Qian. TRAIL: Trace reasoning and agentic issue localization. *arXiv preprint arXiv:2505.08638*, 2025.
- [17] L. Ding. AdaRubric: Task-adaptive rubrics for reliable LLM agent evaluation and reward learning. *arXiv preprint arXiv:2603.21362*, 2026. arXiv Comments field: “KnowFM @ ACL 2026”.
- [18] P. Du. Memory for autonomous LLM agents: Mechanisms, evaluation, and emerging frontiers. *arXiv preprint arXiv:2603.07670*, 2026.
- [19] J. Fang, Y. Peng, X. Zhang, Y. Wang, X. Yi, G. Zhang, Y. Xu, B. Wu, S. Liu, Z. Li, Z. Ren, N. Aletras, X. Wang, H. Zhou, and Z. Meng. A comprehensive survey of self-evolving AI agents: A new paradigm bridging foundation models and lifelong agentic systems. *arXiv preprint arXiv:2508.07407*, 2025.
- [20] C. Fernando, D. S. Banarse, H. Michalewski, S. Osindero, and T. Rocktäschel. Promptbreeder: Self-referential self-improvement via prompt evolution. In *Proceedings of the 41st International Conference on Machine Learning*, pages 13481–13544, 2024. arXiv:2309.16797.
- [21] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen. CRITIC: Large language models can self-correct with tool-interactive critiquing. In *International Conference on Learning Representations*, 2024. arXiv:2305.11738.
- [22] J. Gu, X. Jiang, Z. Shi, H. Tan, X. Zhai, C. Xu, W. Li, Y. Shen, S. Ma, H. Liu, S. Wang, K. Zhang, Y. Wang, W. Gao, L. Ni, and J. Guo. A survey on LLM-as-a-judge. *arXiv preprint arXiv:2411.15594*, 2024.
- [23] A. Gunjal, A. Wang, E. Lau, V. Nath, Y. He, B. Liu, and S. Hendryx. Rubrics as rewards: Reinforcement learning beyond verifiable domains. In *International Conference on Learning Representations*, 2026. arXiv:2507.17746.
- [24] H. Hashemi, J. Eisner, C. Rosset, B. V. Durme, and C. Kedzie. LLM-Rubric: A multidimensional, calibrated approach to automated evaluation of natural language texts. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 13806–13834, 2024. arXiv:2501.00274.

- [25] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou. Large language models cannot self-correct reasoning yet. In *International Conference on Learning Representations*, 2024. arXiv:2310.01798.
- [26] Z. Huang, Y. Zhuang, G. Lu, Z. Qin, H. Xu, T. Zhao, R. Peng, J. Hu, Z. Shen, X. Hu, X. Gu, P. Tu, J. Liu, W. Chen, Y. Fu, Z. Fan, Y. Gu, Y. Wang, Z. Yang, J. Li, and J. Zhao. Reinforcement learning with rubric anchors. *arXiv preprint arXiv:2508.12790*, 2025.
- [27] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *International Conference on Learning Representations*, 2024. arXiv:2310.06770.
- [28] S. Kadavath, T. Conerly, A. Askell, T. Henighan, D. Drain, E. Perez, N. Schiefer, Z. Hatfield-Dodds, N. DasSarma, E. Tran-Johnson, S. Johnston, S. El-Showk, A. Jones, N. Elhage, T. Hume, A. Chen, Y. Bai, S. Bowman, S. Fort, D. Ganguli, D. Hernandez, J. Jacobson, J. Kernion, S. Kravec, L. Lovitt, K. Ndousse, C. Olsson, S. Ringer, D. Amodei, T. Brown, J. Clark, N. Joseph, B. Mann, S. McCandlish, C. Olah, and J. Kaplan. Language models (mostly) know what they know. *arXiv preprint arXiv:2207.05221*, 2022.
- [29] R. Kamoi, Y. Zhang, N. Zhang, J. Han, and R. Zhang. When can LLMs actually correct their own mistakes? A critical survey of self-correction of LLMs. *Transactions of the Association for Computational Linguistics*, 12:1417–1440, 2024. arXiv:2406.01297.
- [30] S. Kapoor, B. Stroebel, Z. S. Siegel, N. Nadgir, and A. Narayanan. AI agents that matter. *Transactions on Machine Learning Research*, 2025. arXiv:2407.01502.
- [31] S. Kapoor, B. Stroebel, P. Kirgis, N. Nadgir, Z. S. Siegel, B. Wei, T. Xue, Z. Chen, F. Chen, S. Utpala, F. Ndzomga, D. Oruganty, S. Luskin, K. Liu, B. Yu, A. Arora, D. Hahm, H. Trivedi, H. Sun, J. Lee, T. Jin, Y. Mai, Y. Zhou, Y. Zhu, R. Bommasani, D. Kang, D. Song, P. Henderson, Y. Su, P. Liang, and A. Narayanan. Holistic agent leaderboard: The missing infrastructure for AI agent evaluation. In *International Conference on Learning Representations*, 2026. arXiv:2510.11977.
- [32] O. Khattab, A. Singhvi, P. Maheshwari, Z. Zhang, K. Santhanam, S. Vardhamanan, S. Haq, A. Sharma, T. T. Joshi, H. Moazam, H. Miller, M. Zaharia, and C. Potts. DSPy: Compiling declarative language model calls into state-of-the-art pipelines. In *International Conference on Learning Representations*, 2024. arXiv:2310.03714 (arXiv version titled “DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines”).
- [33] S. Kim, J. Shin, Y. Cho, J. Jang, S. Longpre, H. Lee, S. Yun, S. Shin, S. Kim, J. Thorne, and M. Seo. Prometheus: Inducing fine-grained evaluation capability in language models. In *International Conference on Learning Representations*, 2024. arXiv:2310.08491.
- [34] S. S. Li, R. Xin, T. Xiao, Y. Wang, R. Shao, Z. Hao, M. Sclar, S. Oh, F. Brahman, P. W. Koh, and Y. Tsvetkov. EvoLM: Self-evolving language models through co-evolved discriminative rubrics. *arXiv preprint arXiv:2605.03871*, 2026.
- [35] S. Lifshitz, S. A. McIlraith, and Y. Du. Multi-agent verification: Scaling test-time compute with multiple verifiers. In *Conference on Language Modeling*, 2025. arXiv:2502.20379.
- [36] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe. Let’s verify step by step. In *International Conference on Learning Representations*, 2024. arXiv:2305.20050.
- [37] T. Liu, R. Xu, T. Yu, I. Hong, C. Yang, T. Zhao, and H. Wang. OpenRubrics: Towards scalable synthetic rubric generation for reward modeling and LLM alignment. In *Proceedings of the 64th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 17417–17437, 2026. arXiv:2510.07743.
- [38] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang. Agent-Bench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024. arXiv:2308.03688.

- [39] Y. Liu, D. Iter, Y. Xu, S. Wang, R. Xu, and C. Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 2511–2522, 2023. arXiv:2303.16634.
- [40] Y. Liu, T. Yang, S. Huang, Z. Zhang, H. Huang, F. Wei, W. Deng, F. Sun, and Q. Zhang. Calibrating LLM-based evaluator. In *Proceedings of the 2024 Joint International Conference on Computational Linguistics, Language Resources and Evaluation (LREC-COLING 2024)*, pages 2638–2656, 2024. arXiv:2309.13308.
- [41] X. H. Lù, A. Kazemnejad, N. Meade, A. Patel, D. Shin, A. Zambrano, K. Stańczak, P. Shaw, C. J. Pal, and S. Reddy. AgentRewardBench: Evaluating automatic evaluations of web agent trajectories. In *Conference on Language Modeling*, 2025. arXiv:2504.08942.
- [42] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhume, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2303.17651.
- [43] L. Madaan, A. K. Singh, R. Schaeffer, A. Poulton, S. Koyejo, P. Stenatorp, S. Narang, and D. Hupkes. Quantifying variance in evaluation benchmarks. *arXiv preprint arXiv:2406.10229*, 2024.
- [44] L. Mei, J. Yao, Y. Ge, Y. Wang, B. Bi, Y. Cai, J. Liu, M. Li, Z.-Z. Li, D. Zhang, C. Zhou, J. Mao, T. Xia, J. Guo, and S. Liu. A survey of context engineering for large language models. *arXiv preprint arXiv:2507.13334*, 2025.
- [45] E. Miller. Adding error bars to evals: A statistical approach to language model evaluations. *arXiv preprint arXiv:2411.00640*, 2024.
- [46] S. Min, K. Krishna, X. Lyu, M. Lewis, W. tau Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi. FActScore: Fine-grained atomic evaluation of factual precision in long form text generation. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 12076–12100, 2023. arXiv:2305.14251.
- [47] H. Mohammadi. trajectory-judge: What outcome-only LLM judges miss on agent trajectories. *arXiv preprint arXiv:2609.00038*, 2026.
- [48] K. Opsahl-Ong, M. J. Ryan, J. Purtell, D. Broman, C. Potts, M. Zaharia, and O. Khattab. Optimizing instructions and demonstrations for multi-stage language model programs. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 9340–9366, 2024. arXiv:2406.11695.
- [49] S. Ouyang, J. Yan, I.-H. Hsu, Y. Chen, K. Jiang, Z. Wang, R. Han, L. T. Le, S. Daruki, X. Tang, V. Tirumalashetty, G. Lee, M. Rofouei, H. Lin, J. Han, C.-Y. Lee, and T. Pfister. ReasoningBank: Scaling agent self-evolving with reasoning memory. In *International Conference on Learning Representations*, 2026. arXiv:2509.25140.
- [50] C. Packer, S. Wooders, K. Lin, V. Fang, S. G. Patil, I. Stoica, and J. E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [51] J. Pan, Y. Zhang, N. Tomlin, Y. Zhou, S. Levine, and A. Suhr. Autonomous evaluation and refinement of digital agents. In *Conference on Language Modeling*, 2024. arXiv:2404.06474.
- [52] A. Panickssery, S. R. Bowman, and S. Feng. LLM evaluators recognize and favor their own generations. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2404.13076.
- [53] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023. arXiv:2304.03442.
- [54] D. Quinn, D. Dylan, R. Healy, F. Carroll, M. Donnelly, and C. Sheehan. Inducing reward-free judging rubrics that reduce over-crediting in agent evaluation. *arXiv preprint arXiv:2608.13564*, 2026.
- [55] S. Shankar, J. D. Zamfirescu-Pereira, B. Hartmann, A. G. Parameswaran, and I. Arawjo. Who validates the validators? Aligning LLM-assisted evaluation of LLM outputs with human preferences. In *Proceedings of the 37th Annual ACM Symposium on User Interface Software and Technology*, 2024. doi: 10.1145/3654777.3676450. arXiv:2404.12272.

- [56] W. Shi, Y. Wang, Y. Zhao, Y. Chen, F. Feng, X. Hao, X. Su, Q. Gu, H. Su, X. Cai, and X. He. AJ-Bench: Benchmarking Agent-as-a-Judge for environment-aware evaluation. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 25371–25413, 2026. arXiv:2604.18240.
- [57] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. arXiv:2303.11366.
- [58] C. Snell, J. Lee, K. Xu, and A. Kumar. Scaling LLM test-time compute optimally can be more effective than scaling parameters for reasoning. In *International Conference on Learning Representations*, 2025. arXiv:2408.03314.
- [59] H. Su, R. Sun, J. Yoon, P. Yin, T. Yu, and S. Ö. Arik. Learn-by-interact: A data-centric framework for self-adaptive agents in realistic environments. In *International Conference on Learning Representations*, 2025. arXiv:2501.10893.
- [60] M. Suzgun, M. Yuksekogonul, F. Bianchi, D. Jurafsky, and J. Zou. Dynamic cheatsheet: Test-time learning with adaptive memory. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 7080–7106, 2026. arXiv:2504.07952.
- [61] Z. Tian, J. Zhang, R. Li, X. Bo, Y. Li, and X. Chen. ARCO: Adaptive rubrics with co-evolution for multi-step LLM-based agents. *arXiv preprint arXiv:2606.21262*, 2026.
- [62] H. Trivedi, T. Khot, M. Hartmann, R. Manku, V. Dong, E. Li, S. Gupta, A. Sabharwal, and N. Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 16022–16076, 2024. arXiv:2407.18901.
- [63] P. Verga, S. Hofstätter, S. Althammer, Y. Su, A. Piktus, A. Arkhangorodsky, M. Xu, N. White, and P. Lewis. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *arXiv preprint arXiv:2404.18796*, 2024.
- [64] V. Viswanathan, Y. Sun, S. Ma, X. Kong, M. Cao, G. Neubig, and T. Wu. Checklists are better than reward models for aligning language models. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2507.18624.
- [65] Y. Wan, T. Fang, Z. Li, Y. Huo, W. Wang, H. Mi, D. Yu, and M. R. Lyu. Inference-time scaling of verification: Self-evolving deep research agents via test-time rubric-guided verification. In *Findings of the Association for Computational Linguistics: ACL 2026*, pages 24822–24835, 2026. arXiv:2601.15808.
- [66] B. Wang, W. Su, H. Tian, H. Kong, T. Yang, T. Yao, Q. Pan, Y. Wu, Q. Ai, M. Zhang, and Y. Liu. Co-evolving LLM evaluators and policies via DynamicRubric. *arXiv preprint arXiv:2607.20083*, 2026.
- [67] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024. arXiv:2305.16291.
- [68] T. Wang, I. Kulikov, O. Golovneva, P. Yu, W. Yuan, J. Dwivedi-Yu, R. Y. Pang, M. Fazel-Zarandi, J. Weston, and X. Li. Self-taught evaluators. *arXiv preprint arXiv:2408.02666*, 2024.
- [69] X. Wang, J. Wei, D. Schuurmans, Q. V. Le, E. H. Chi, S. Narang, A. Chowdhery, and D. Zhou. Self-consistency improves chain of thought reasoning in language models. In *International Conference on Learning Representations*, 2023. arXiv:2203.11171.
- [70] Z. Wang and E. Blanco. Generating and refining dynamic evaluation rubrics for LLM-as-a-Judge. *arXiv preprint arXiv:2605.30568*, 2026.
- [71] Z. Z. Wang, J. Mao, D. Fried, and G. Neubig. Agent workflow memory. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 63897–63911, 2025. arXiv:2409.07429.
- [72] T. Wei, N. Sachdeva, B. Coleman, Z. He, Y. Bei, X. Ning, M. Ai, Y. Li, J. He, E. H. Chi, C. Wang, S. Chen, F. Pereira, W.-C. Kang, and D. Z. Cheng. Evo-Memory: Benchmarking LLM agent test-time learning with self-evolving memory. *arXiv preprint arXiv:2511.20857*, 2025.

- [73] W. Xu, G. Zhu, X. Zhao, L. Pan, L. Li, and W. Y. Wang. Pride and prejudice: LLM amplifies self-bias in self-refinement. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 15474–15492, 2024. arXiv:2402.11436.
- [74] W. Xu, Z. Liang, K. Mei, H. Gao, J. Tan, and Y. Zhang. A-MEM: Agentic memory for LLM agents. In *Advances in Neural Information Processing Systems*, volume 38, 2025. arXiv:2502.12110.
- [75] T. Xue, W. Qi, T. Shi, C. H. Song, B. Gou, D. Song, H. Sun, and Y. Su. An illusion of progress? Assessing the current state of web agents. In *Conference on Language Modeling*, 2025. arXiv:2504.01382.
- [76] C. Yang, X. Wang, Y. Lu, H. Liu, Q. V. Le, D. Zhou, and X. Chen. Large language models as optimizers. In *International Conference on Learning Representations*, 2024. arXiv:2309.03409.
- [77] L. Yang, Z. Yu, T. Zhang, S. Cao, M. Xu, W. Zhang, J. E. Gonzalez, and B. Cui. Buffer of thoughts: Thought-augmented reasoning with large language models. In *Advances in Neural Information Processing Systems*, 2024. arXiv:2406.04271.
- [78] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In *Advances in Neural Information Processing Systems*, 2023. arXiv:2305.10601.
- [79] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*, 2023. arXiv:2210.03629.
- [80] S. Yao, N. Shinn, P. Razavi, and K. Narasimhan. τ -bench: A benchmark for tool-agent-user interaction in real-world domains. In *International Conference on Learning Representations*, 2025. arXiv:2406.12045.
- [81] W. Yuan, R. Y. Pang, K. Cho, X. Li, S. Sukhbaatar, J. Xu, and J. Weston. Self-rewarding language models. In *Proceedings of the 41st International Conference on Machine Learning*, pages 57905–57923, 2024. arXiv:2401.10020.
- [82] M. Yuksekgonul, F. Bianchi, J. Boen, S. Liu, P. Lu, Z. Huang, C. Guestrin, and J. Zou. Optimizing generative AI by backpropagating language model feedback. *Nature*, 639:609–616, 2025. arXiv:2406.07496 (arXiv version titled “TextGrad: Automatic ‘Differentiation’ via Text”).
- [83] L. Zhang, A. Hosseini, H. Bansal, M. Kazemi, A. Kumar, and R. Agarwal. Generative verifiers: Reward modeling as next-token prediction. In *International Conference on Learning Representations*, 2025. arXiv:2408.15240.
- [84] Q. Zhang, C. Hu, S. Upasani, B. Ma, F. Hong, V. Kamanuru, J. Rainton, C. Wu, M. Ji, H. Li, U. Thakker, J. Zou, and K. Olukotun. Agentic context engineering: Evolving contexts for self-improving language models. In *International Conference on Learning Representations*, 2026. arXiv:2510.04618.
- [85] X. Zhang, G. Wang, Y. Cui, Z. Li, W. Qiu, B. Zhu, and P. He. Who grades the grader? co-evolving evaluation metrics and skills for self-improving LLM agents. *arXiv preprint arXiv:2607.12790*, 2026.
- [86] Y. Zhang, Y. Li, A. Payani, and L. Wang. AdaMEM: Test-time adaptive memory for language agents. In *Proceedings of the 43rd International Conference on Machine Learning*, 2026. arXiv:2606.05684.
- [87] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen. A survey on the memory mechanism of large language model based agents. *arXiv preprint arXiv:2404.13501*, 2024.
- [88] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang. ExpeL: LLM agents are experiential learners. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19632–19642, 2024. arXiv:2308.10144.
- [89] X. Zhao, Z. Kang, A. Feng, S. Levine, and D. Song. Learning to reason without external rewards. In *International Conference on Learning Representations*, 2026. arXiv:2505.19590.
- [90] L. Zheng, W.-L. Chiang, Y. Sheng, S. Zhuang, Z. Wu, Y. Zhuang, Z. Lin, Z. Li, D. Li, E. P. Xing, H. Zhang, J. E. Gonzalez, and I. Stoica. Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems*, 2023. Datasets and Benchmarks Track. arXiv:2306.05685.

- [91] W. Zhong, L. Guo, Q. Gao, H. Ye, and Y. Wang. MemoryBank: Enhancing large language models with long-term memory. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 19724–19731, 2024. arXiv:2305.10250.
- [92] A. Zhou, K. Yan, M. Shlapentokh-Rothman, H. Wang, and Y.-X. Wang. Language agent tree search unifies reasoning, acting, and planning in language models. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pages 62138–62160. PMLR, 2024. arXiv:2310.04406.
- [93] H. Zhou, Y. Chen, S. Guo, X. Yan, K. H. Lee, Z. Wang, K. Y. Lee, G. Zhang, K. Shao, L. Yang, and J. Wang. Memento: Fine-tuning LLM agents without fine-tuning LLMs. *arXiv preprint arXiv:2508.16153*, 2025.
- [94] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig. WebArena: A realistic web environment for building autonomous agents. In *International Conference on Learning Representations*, 2024. arXiv:2307.13854.
- [95] Y. Zhou, A. I. Muresanu, Z. Han, K. Paster, S. Pitis, H. Chan, and J. Ba. Large language models are human-level prompt engineers. In *International Conference on Learning Representations*, 2023. arXiv:2211.01910.
- [96] K. Zhu, H. Li, S. Wu, T. Xing, D. Ma, X. Tang, M. Liu, J. Yang, J. Liu, Y. E. Jiang, C. Zhang, C. Lin, J. Wang, G. Zhang, and W. Zhou. Scaling test-time compute for LLM agents. *arXiv preprint arXiv:2506.12928*, 2025.
- [97] M. Zhuge, C. Zhao, D. R. Ashley, W. Wang, D. Khizbullin, Y. Xiong, Z. Liu, E. Chang, R. Krishnamoorthi, Y. Tian, Y. Shi, V. Chandra, and J. Schmidhuber. Agent-as-a-Judge: Evaluate agents with agents. In *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pages 80569–80611. PMLR, 2025. arXiv:2410.10934.
- [98] Y. Zuo, K. Zhang, L. Sheng, S. Qu, G. Cui, X. Zhu, H. Li, Y. Zhang, X. Long, E. Hua, B. Qi, Y. Sun, Z. Ma, L. Yuan, N. Ding, and B. Zhou. TTRL: Test-time reinforcement learning. In *Advances in Neural Information Processing Systems*, 2025. arXiv:2504.16084.

A Calibration outcomes

Table 2: Warm-up-v2 calibration outcome per world (campaign A). Agreement of the authored suite (S_0) and of the calibrated proposal with the four gate labels, and the frozen result.

World	S_0 gate agr.	Proposal gate agr.	Decision	Frozen agg., τ
tau2 airline	100.0%	75.0%	rejected, S_0 kept	min, 0.99
tau2 retail	66.7%	83.3%	accepted	min, 0.99
tau2 telecom	100.0%	100.0%	no improvement possible, S_0 kept	weighted, 0.85
AppWorld normal	100.0%	100.0%	no improvement possible, S_0 kept	min, 0.99
AppWorld challenge	100.0%	100.0%	no improvement possible, S_0 kept	weighted, 0.85

B Authored suites

The calibrated suites deployed in campaign D contain:

- **airline**: two judges, `final_state_outcome_verifier` and `policy_and_confirmation_guard`; min, $\tau = 0.99$;
- **retail**: one judge, `state_outcome_verifier`; min, 0.99 (calibration dropped a policy-compliance guard);
- **telecom**: three judges, `final_mms_outcome_verifier`, `policy_and_precondition_guard` and `diagnostic_coverage_judge`; weighted, 0.85;
- **AppWorld normal**: two judges, `final_state_outcome_verifier` and `unintended_side_effect_guard`; min, 0.99.

Fresh suites in the single-run GPT-5.5 study contained two (retail, telecom, AppWorld normal) or three (airline) judges, with min or weighted aggregation and thresholds of 0.85 to 0.99. Every judge is conditioned on the state read-back.

C Full matched-campaign table and the label-budget grid

Table 3: All arms of campaign A (GPT-5.5, five matched repeats). “Oracle” rows read the true label on every task and are not deployable. “Collapsing” and “delta” are our re-implementation of ACE, with and without a full playbook rewrite after every task; rows labelled plain “ACE” use the memory machinery released by the ACE authors. Warm playbooks are fitted from the ten warm-up labels.

Arm	Signal	Tau2	AppWorld	Overall
Plain agent	none	74.4 $\pm$ 5.4	78.4 $\pm$ 9.6	76.4 $\pm$ 6.2
ACE re-impl. collapsing, cold	self-graded	72.8 $\pm$ 10.7	90.4 $\pm$ 3.6	81.6 $\pm$ 4.6
ACE re-impl. collapsing, cold	calibrated evaluator	72.8 $\pm$ 9.6	92.0 $\pm$ 4.0	82.4 $\pm$ 3.0
ACE re-impl. collapsing, cold	GT oracle	72.0 $\pm$ 4.9	88.8 $\pm$ 5.2	80.4 $\pm$ 3.6
ACE re-impl. collapsing, warm, frozen	none	72.8 $\pm$ 7.2	88.8 $\pm$ 5.2	80.8 $\pm$ 4.6
ACE re-impl. collapsing, warm	self-graded	69.6 $\pm$ 8.3	88.0 $\pm$ 7.5	78.8 $\pm$ 6.1
ACE re-impl. collapsing, warm	calibrated evaluator	72.8 $\pm$ 4.4	92.8 $\pm$ 5.2	82.8 $\pm$ 2.3
ACE re-impl. delta, cold	self-graded	75.2 $\pm$ 6.6	92.0 $\pm$ 4.0	83.6 $\pm$ 4.1
ACE re-impl. delta, cold	calibrated evaluator	75.2 $\pm$ 12.5	91.2 $\pm$ 4.4	83.2 $\pm$ 4.8
ACE re-impl. delta, cold	GT oracle	69.6 $\pm$ 11.2	91.2 $\pm$ 3.4	80.4 $\pm$ 6.7
ACE, cold	self-graded	71.2 $\pm$ 8.7	91.2 $\pm$ 3.4	81.2 $\pm$ 3.6
ACE, warm	self-graded	77.6 $\pm$ 6.1	90.4 $\pm$ 3.6	84.0 $\pm$ 3.2
ACE, warm	calibrated evaluator	76.8 $\pm$ 4.4	92.0 $\pm$ 4.9	84.4 $\pm$ 2.6
Windowed rubric co-evolution (K=20)	evolving suite	74.4 $\pm$ 4.6	88.0 $\pm$ 5.7	81.2 $\pm$ 3.0
SELSUITE, fresh suite	frozen suite	78.4 $\pm$ 7.8	86.4 $\pm$ 2.2	82.4 $\pm$ 3.9
SELSUITE + calibrated suite	frozen suite	79.2 $\pm$ 5.9	89.6 $\pm$ 3.6	84.4 $\pm$ 4.6

Three readings of Table 3 inform §8. Grading ACE’s deployment trajectories with the calibrated FDE suite instead of trace self-inference, on top of the warm playbook, changes nothing measurable (84.4 $\pm$ 2.6 versus 84.0 $\pm$ 3.2 overall; paired +0.4 $\pm$ 2.6 pp, [−2.8, +3.6]). Giving ACE the true label on every task does not help (80.4 on both memory rows, below the label-free rows): once lessons are discarded by the per-task rewrite, no signal can matter, and disabling the rewrite (delta) is worth +2.0 $\pm$ 2.0 pp with the same signal. On the collapsing implementation the calibrated evaluator’s role is to prevent drift: warm-then-frozen 72.8, warm with label-free adaptation 69.6 (−3.2 $\pm$ 3.4 paired), warm with calibrated-evaluator adaptation 72.8 on tau2.

D Exact values behind Figures 4 and 5

Table 4: Operating point of the calibrated suites (campaign D; mean over five deployments on the whole task sets), values plotted in Figure 4. Judges and aggregation as frozen after calibration; per-attempt false-positive (FP) and false-negative (FN) rates of the suite verdict against ground truth; discrimination = mean judge score on GT-pass minus GT-fail attempts; retry gain = deployed minus first-attempt success; oracle = success if the better of the two attempts had always been selected.

World	Judges, agg., τ	Judged pass	GT	Retried	FP	FN	Discr.	Retry gain	Oracle
Airline (40)	2, min, 0.99	67.5%	64.0	37.0%	39%	26%	+0.18	+1.5	66.0
Retail (104)	1, min, 0.99	86.5%	84.2	26.9%	53%	25%	+0.13	+1.0	88.5
Telecom (104)	3, weighted, 0.85	72.3%	79.6	38.8%	13%	23%	+0.38	+4.6	84.2
AppWorld normal (158)	2, min, 0.99	84.7%	90.8	20.0%	29%	23%	+0.43	+1.5	91.1

Table 5: Two-model single-run study (campaign C), values plotted in Figure 5. Success (%) on the full suites; one run per cell. **Bold** = best in row.

Model	World	Plain agent	ACE, no GT	SELSUITE
GPT-5.5 @low	AppWorld normal (168)	87.5	90.5	91.7
	AppWorld challenge (417)	64.5	71.2	77.7
	tau2 airline (50)	64.0	70.0	66.0
	tau2 retail (114)	83.3	83.3	83.3
	tau2 telecom (114)	70.2	69.3	75.4
Opus-4.8 @off	AppWorld normal (168)	70.8	86.9	92.3
	tau2 airline (50)	68.0	70.0	66.0
	tau2 retail (114)	82.5	85.1	85.1
	tau2 telecom (114)	62.3	69.3	73.7